# Code Test

## Objective

The purpose of this assessment is to evaluate your fundamental backend development skills, including code structure, API development, database design, error handling, code quality, and problem-solving approach.

We are not only evaluating whether the application produces the correct result. We will also review how the solution is designed and implemented.

## Technology Stack

Please use the following technologies:

- **Framework:** NestJS
- **Language:** Node.js + TypeScript
- **Database:** MySQL 8
- **Cache Service:** Redis

---

## Functional Requirements

### 1. Create an API Endpoint

Create a REST API endpoint using the **POST** method.

The endpoint should accept the following two parameters:

- `id1`
- `id2`

You may define an appropriate API path and request body structure.

Example request:

```
{
  "id1": "ABC123",
  "id2": "XYZ456"
}
```

Both `id1` and `id2` are required.

The application should validate the request and return an appropriate error response when the input is invalid or required parameters are missing.

---

## 2. Database

Create an appropriate table in MySQL 8 to store:

- `id1`
- `id2`
- `userID`

You may include additional fields that you consider appropriate.

The combination of `id1` and `id2` should identify one record.

---

## 3. Processing Logic

When the API receives a valid request:

- Check the database to determine whether a record containing the same combination of `id1` and `id2` already exists.
- If the record **exists**:
  - Retrieve the existing `userID`.
  - Return the `userID` in JSON format.
- If the record **does not exist**:
  - Generate a new `userID` using **UUID v4**.
  - Store the `id1`, `id2`, and generated `userID` in the database.
  - Return the generated `userID` in JSON format.

## Expected Behaviour

Calling the API multiple times with the same `id1` and `id2` combination should return the **same** `userID`.

For example:

First request:

```
{
  "id1": "ABC123",
  "id2": "XYZ456"
}
```

Response:

```
{
  "userID": "550e8400-e29b-41d4-a716-446655440000"
}
```

A later request with the same `id1` and `id2` should return the same `userID`.

---

## Redis

Redis is included in the technology stack.

You may decide how Redis can appropriately be used in this application.

If Redis is used, please briefly explain its purpose in the README.

The application should continue to maintain correct data in MySQL and should not rely on Redis as the permanent source of data.

---

# Minimum Requirements

The following are considered the minimum requirements for a passing submission.

### 1. Functional Correctness

- The application can be started and tested successfully.
- The POST API accepts `id1` and `id2`.
- A new UUID v4 `userID` is generated when the combination does not exist.
- The record is correctly stored in MySQL.
- Repeating the same `id1` and `id2` returns the existing `userID`.
- Invalid requests are handled appropriately.

- Unexpected application/database errors do not cause uncontrolled application crashes.

## 2. Code Quality

The submitted code should demonstrate reasonable software development practices, including:

- Clear and understandable code
- Meaningful naming of variables, methods, classes, modules, and files
- Appropriate separation of responsibilities
- Avoidance of unnecessary duplicated code
- Appropriate use of TypeScript types
- Consistent formatting and coding style
- No unnecessary or unused code

Candidates are encouraged to follow standard NestJS project structure and conventions.

## 3. Database Design

The database implementation should demonstrate:

- Appropriate data types
- Appropriate constraints and indexes
- Protection against unintended duplicate records for the same `id1` and `id2` combination
- Proper use of database queries or ORM/database framework

## 4. Error Handling & Validation

The application should:

- Validate required request parameters
- Return appropriate HTTP status codes
- Return understandable error responses
- Handle database or application errors appropriately

- Avoid exposing unnecessary internal technical information in API error responses

### 5. Security & Configuration

- Database passwords, Redis passwords, API keys, or other secrets must **not** be hard-coded in the source code.
- Configuration should be provided through environment variables or another appropriate configuration mechanism.
- Sensitive credentials must not be committed to the Git repository.

### 6. Git Usage

The project must be submitted through a Git repository.

We will review:

- Repository structure
- Commit history
- Meaningful commit messages
- Whether unnecessary generated files, dependencies, credentials, or local configuration files have been committed

---

# Evaluation Criteria

The submission will be evaluated approximately according to the following areas:

| Area | Weight |
|---|---|
| Functional correctness | 30% |
| Code quality and project structure | 20% |
| Database design | 15% |
| Error handling and validation | 15% |
| Testing | 10% |
| Documentation, Git usage and general engineering practices | 10% |

A submission should generally achieve **60% or above** and satisfy the core functional requirements to be considered a passing submission.

However, serious issues such as committed credentials, an application that cannot run, or failure to implement the main required functionality may result in the submission being considered unsuccessful regardless of the numerical score.

---

# Bonus Points

The following are **not mandatory** for passing the assessment, but may receive additional consideration.

## 1. Automated Testing

Provide meaningful unit and/or integration tests covering important scenarios, for example:

- New `id1` + `id2`
- Existing `id1` + `id2`
- Invalid input
- Error scenarios

## 2. Docker

Provide a `Dockerfile` and/or `docker-compose.yml` that allows the application and its dependencies to be started easily.

For example:

- NestJS application
- MySQL 8
- Redis

## 3. Sequence Diagram

Provide a sequence diagram showing the main processing flow of the application.

## 4. API Documentation

Provide API documentation using Swagger/OpenAPI or another appropriate method.

### 5. Concurrency Handling

Consider what could happen if two requests containing the same new `id1` and `id2` arrive at approximately the same time.

Implement an appropriate solution to prevent inconsistent or duplicate data.

Please briefly explain your approach in the README.

---

# Submission Requirements

Please submit the completed project through a **Git repository**.

The repository should include a `README.md` containing:

- Project overview
- Prerequisites
- Installation instructions
- Environment/configuration instructions
- Database setup instructions
- Instructions for starting the application
- API endpoint and example request
- Instructions for running tests, if provided
- Any assumptions or important technical decisions

Please provide an `.env.example` or equivalent configuration example where appropriate.

**Do not include real passwords, credentials, API keys, or other secrets in the repository.**

---

# Notes

You may use third-party libraries where appropriate.

You are free to make reasonable technical decisions that are not explicitly specified in this document. If you make an important design decision or assumption, please briefly explain it in the README.

We value **clear, reliable and maintainable code** more than unnecessary complexity.